University of Eloued

Faculty of Exact Sciences
Department of Computer science
For Second-year Master2: IA & Data Science

Rapport de TP02 : Traitement des Big Data avec Pandas, Pandas Chunking et Dask

Étudiante : Akila Saker
Module : Big Data Analytics

Titre du TP : Traitement et comparaison des performances sur un fichier CSV (+5GB) volumineux

Année universitaire : 2026–2027

1.Intoduction

Le développement des technologies numériques entraîne la production de volumes de données de plus en plus importants. Le traitement de ces données nécessite des méthodes capables de gérer efficacement le temps d'exécution et la consommation des ressources matérielles, notamment la mémoire vive (RAM).

Python propose plusieurs bibliothèques pour traiter les données, parmi lesquelles Pandas et Dask. Pandas est largement utilisé pour la manipulation des données tabulaires, tandis que Dask permet de répartir les calculs sur plusieurs partitions et de traiter de grands volumes de données.

Dans ce TP, nous étudions trois approches : le chargement classique avec Pandas, le traitement par blocs avec Pandas Chunking et le traitement avec Dask. Nous comparons leurs performances sur un fichier CSV volumineux.

2. Objectifs du TP

Les principaux objectifs sont les suivants :

- Créer ou utiliser un fichier CSV de grande taille, supérieur à 5 GB.
- Tester le chargement complet du fichier avec Pandas.
- Traiter progressivement les données avec Pandas Chunking.
- Utiliser Dask pour traiter le fichier par partitions.
- Mesurer et comparer le temps d'exécution et la consommation de mémoire RAM.
- Identifier les avantages et les limites de chaque méthode.
- Déterminer la méthode la plus adaptée à un environnement disposant de ressources mémoire limitées.

3. Environnement de travail

Le projet a été réalisé avec Python et les bibliothèques de traitement de données suivantes :

- **Pandas :** lecture et manipulation des fichiers CSV.
- **Dask :** traitement de données volumineuses par partitions.
- **Matplotlib :** création des graphiques comparatifs.
- **NumPy :** manipulation de valeurs numériques pour la visualisation.
- **OS :** gestion des chemins de fichiers et des dossiers de résultats.

Les mesures ont été effectuées dans l'environnement d'exécution disponible. La quantité de mémoire RAM disponible constitue une contrainte importante pour la méthode classique.

4. Description du jeu de données

Le fichier principal utilisé dans l'expérience est `big_synthetic.csv`. Il s'agit d'un jeu de données synthétiques générées artificiellement avec Python pour simuler un volume important de données.

Les caractéristiques du fichier sont les suivantes :

Caractéristique	Valeur
Nom du fichier	`big_synthetic.csv`
Format	CSV
Taille rapportée	6,009 GB
Nombre de lignes	49 300 000

Origine	Données synthétiques générées par Python

Les données synthétiques permettent de tester les capacités des méthodes de traitement sans nécessiter une collecte réelle de données.

Remarque : un fichier de test distinct, `test_data.csv`, a également été utilisé pendant les essais préliminaires. Il contient 10 000 lignes et trois colonnes : `temperature`, `humidity` et `sensor_id`. Ce petit fichier sert à vérifier les opérations de lecture et de calcul ; il ne doit pas être confondu avec le fichier principal de 6,009 GB.

5. Méthodologie

5.1 : Standard Pandas

La méthode classique utilise la fonction `pd.read_csv()` pour charger le fichier CSV entier dans un DataFrame.

Exemple :

```
import pandas as pd

df = pd.read_csv("data/big_synthetic.csv")

print(df.shape)
print(df.head())
```

Cette approche est simple et pratique pour les fichiers de taille modérée. Cependant, un fichier volumineux peut nécessiter beaucoup plus de mémoire que sa taille sur disque, en fonction des types de données et des opérations effectuées.

Résultat de notre expérience : l'exécution n'a pas démarré, car les conditions de sécurité relatives à la mémoire disponible n'étaient pas satisfaites. Aucun temps d'exécution ni pic de RAM n'a donc été mesuré pour cette méthode.

Il ne s'agit pas d'une erreur `MemoryError` confirmée : le programme a été empêché de démarrer par le contrôle de sécurité.

5.2: Pandas Chunking

Pandas Chunking consiste à lire le fichier en plusieurs blocs au lieu de charger toutes les lignes simultanément.

La fonction utilisée est :

```
pd.read_csv("data/big_synthetic.csv", chunksize=100000)
```

La taille des blocs a été fixée à 100 000 lignes. Chaque bloc peut être traité séparément, ce qui permet d'effectuer des calculs progressifs et de limiter la quantité de données conservée simultanément en mémoire.

Cette méthode est particulièrement utile lorsque la mémoire RAM est limitée.

5. 3 : Dask

Dask est une bibliothèque qui permet de travailler avec des volumes de données importants grâce à la division du travail en partitions et à l'évaluation différée des calculs.

Exemple de lecture :

```
import dask.dataframe as dd

df = dd.read_csv("data/big_synthetic.csv")

print(df.npartitions)

result = df.compute()
```

Dans une utilisation réelle, il est préférable de calculer directement les agrégations nécessaires plutôt que de convertir systématiquement l'ensemble du fichier en un seul DataFrame Pandas.

Dask peut améliorer les performances de certaines opérations, mais la gestion des partitions et l'exécution des tâches consomment également des ressources.

6. Résultats expérimentaux

Après l'exécution des méthodes, les résultats enregistrés sont les suivants :

Méthode	État	Taille (GB)	Nombre de lignes	Temps (s)	Pic RAM (MB)
Standard Pandas	Non démarré : RAM insuffisante	6,009	49 300 000	N/A	N/A
Pandas Chunking	Terminé	6,009	49 300 000	115,407813	78,277344
Dask	Terminé	6,009	49 300 000	111,526876	327,023438

Interprétation du tableau :

- Standard Pandas n'a fourni aucune mesure, car l'exécution n'a pas démarré.
- Pandas Chunking a terminé le traitement en environ 115,41 secondes.
- Dask a terminé le traitement en environ 111,53 secondes.
- Dask a été légèrement plus rapide, tandis que Pandas Chunking a présenté un pic de RAM mesuré plus faible.

Les valeurs N/A indiquent des mesures indisponibles, et non des valeurs égales à zéro.

7. Comparaison graphique

7.1. Comparaison du temps d'exécution

Le graphique `performance_time.png` présente le temps d'exécution des méthodes.

Les résultats mesurés sont :

- Pandas Chunking : 115,41 secondes.
- Dask : 111,53 secondes.
- Standard Pandas : aucune mesure, car l'exécution n'a pas démarré.

La différence de temps entre Chunking et Dask est :

115,407813 − 111,526876 = 3,880937 secondes.

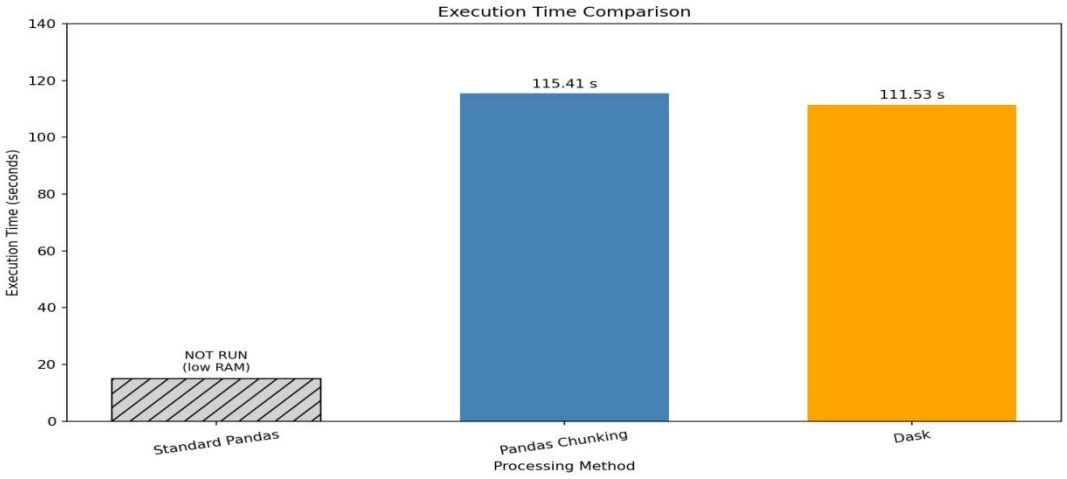
Le gain relatif de Dask par rapport à Chunking est d'environ 3,36 %.

Dask est donc légèrement plus rapide dans cette expérience. Cependant, cette différence reste modérée et dépend de la configuration matérielle et des conditions d'exécution.

Le graphique doit afficher les trois méthodes, mais la catégorie Standard Pandas doit être annotée « N/A – Not started » plutôt que représentée par une mesure fictive.

7.2. Comparaison de la consommation mémoire

Le graphique `performance_ram.png` présente les pics de RAM mesurés Les résultats sont :



.

- Pandas Chunking : 78,277344 MB.
- Dask : 327,023438 MB.
- Standard Pandas : aucune mesure disponible.

La différence entre les pics de RAM de Dask et de Chunking est :

327,023438 − 78,277344 = 248,746094 MB.

Dask a utilisé environ 4,18 fois le pic de RAM mesuré pour Chunking.

Ces valeurs sont les résultats rapportés par le système de mesure utilisé. Elles dépendent notamment de la manière dont la mémoire a été surveillée et ne représentent pas nécessairement toute la mémoire utilisée par le système.

Le graphique doit donc présenter les deux valeurs mesurées et signaler l'absence de mesure pour Standard Pandas.

8. Discussion

Les résultats montrent que chaque méthode présente des avantages et des limites.

Standard Pandas propose une approche simple pour lire et manipuler un fichier CSV. Toutefois, le chargement complet de grands fichiers peut être incompatible avec les ressources disponibles. Dans notre expérience, le contrôle de sécurité a empêché le démarrage du traitement ; nous ne disposons donc pas de mesure permettant de comparer directement cette méthode aux deux autres.

Pandas Chunking a permis de traiter le fichier progressivement. Son pic de RAM mesuré, de 78,28 MB, est le plus faible parmi les méthodes exécutées. Cette méthode constitue une solution intéressante pour les ordinateurs disposant d'une mémoire limitée.

Dask a obtenu le meilleur temps d'exécution, avec 111,53 secondes. Son pic de RAM mesuré était toutefois plus élevé que celui de Chunking. Il convient donc de prendre en compte les ressources matérielles lors du choix de la méthode.

Ces observations concernent uniquement l'expérience réalisée. Elles ne signifient pas que Dask sera systématiquement plus rapide ou que Chunking sera toujours la méthode la moins coûteuse en mémoire dans tous les contextes.

9. Conclusion générale

Ce TP nous a permis d'étudier trois approches de traitement de données volumineuses avec Python : Standard Pandas, Pandas Chunking et Dask.

Le fichier principal contenait 49,3 millions de lignes et sa taille rapportée était de 6,009 GB. La méthode Standard Pandas n'a pas pu démarrer dans les conditions de sécurité retenues, en raison de la mémoire disponible. En revanche, Pandas Chunking et Dask ont terminé le traitement avec succès.

Pandas Chunking a enregistré un temps de 115,41 secondes et un pic de RAM de 78,28 MB. Dask a enregistré un temps de 111,53 secondes et un pic de RAM de 327,02 MB.

Ainsi, Dask a été légèrement plus rapide dans cette expérience, alors que Pandas Chunking a consommé moins de mémoire selon les mesures obtenues.

En conclusion, Pandas Chunking est un choix pertinent lorsque la priorité est de limiter la consommation mémoire, tandis que Dask constitue une alternative intéressante lorsque l'on recherche un temps de traitement légèrement plus court et que les ressources disponibles le permettent. Le choix de la méthode doit dépendre de la taille des données, des objectifs du traitement et des ressources matérielles.

10. Fichiers de résultats

Le dossier `results` contient les fichiers de résultats générés pendant le TP :

- `performance_comparison.csv` : tableau récapitulatif des résultats.
- `performance_time.png` : graphique de comparaison des temps d'exécution.
- `performance_ram.png` : graphique de comparaison des pics de RAM.
- `chunking_results.csv` : résultats du traitement par blocs.
- `dask_results.csv` : résultats du traitement avec Dask.
- `task1_oom_report.txt` : rapport de la tentative Standard Pandas et de l'état de sécurité.

Ces fichiers permettent de vérifier les résultats et de compléter le rapport avec les tableaux et les graphiques produits pendant l'expérience.